

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-a268b104b9e2e5f63.jsonl`  
- SHA-256: `767dccfb2aac85f516e31a8cbd2f04d535c3a40c1c69a86871bd81eebe79e97f`  
- Source modified: `2026-06-08T08:41:03+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_d251ad6c-472`  
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-08T08:39:01.814Z)

Determine the OUTPUT CONTRACT the collector must serve to the indexer eval/training.

Files: C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py (load_golden_gts), C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_local.py (manifest JSON format), C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py (NEW partial-label eval, note its labeled-window concept), C:/Users/avidu/Projects/badminton-highlight-indexer/backend/tools/rally_slicer.py (load_rally_labels), C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/distill_local.py (how labels feed training / label_candidates).

Determine, with anchors:

- The exact CSV columns + manifest JSON the indexer consumes for golden labels.
- The minimal label contract (which columns are load-bearing vs ignored).
- How IoU matching + the labeled-window restriction work, and whether partial labels / a labeled-window are representable in what the collector currently exports.
- What metadata (fps, frame_width, video path, trajectory path) the manifest needs that the collector could supply.

2. assistant (2026-06-08T08:39:04.243Z)

I'll analyze the collector's output contract by examining how the indexer consumes labels across these key files. Let me start with parallel reads to understand the complete pipeline.

3. user (2026-06-08T08:39:04.526Z)

```
1      """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +  
2      "+X%".  
3      Wires the already-tested pieces together:  
4          WASB trajectory CSV (Frame,Visibility,X,Y)  
5          -> TrackNetRunner.trajectory_to_action_windows(cfg)    [predict_fn]  
6          -> backend.eval.calibration (improvement_report / cross_validate)  
7  
8      Run (after a full-video WASB pass exists):  
9          python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \  
10             --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920  
11  
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that  
13      same GT is an **optimistic** upper bound. The trustworthy number is the  
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,  
15      `leave_one_video_out` for precision/F1 generalization).  
16      """  
17      from __future__ import annotations  
18  
19      import csv
```

```

20     import argparse
21     from typing import List, Tuple, Callable
22
23     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
24     from backend.eval.calibration import (
25         select_best, improvement_report, cross_validate, evaluate,
26     )
27
28     Interval = Tuple[float, float]
29     Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
30
31     BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
32
33
34     def load_golden_gts(csv_path: str) -> List[Interval]:
35         """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36         gts: List[Interval] = []
37         with open(csv_path, newline="", encoding="utf-8") as f:
38             reader = csv.DictReader(f)
39             reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
40             for row in reader:
41                 row = {(k or "").strip().lower(): v for k, v in row.items()}
42                 try:
43                     s = float((row.get("start_time") or "").strip())
44                     e = float((row.get("end_time") or "").strip())
45                 except (TypeError, ValueError):
46                     continue
47                 if e > s:
48                     gts.append((s, e))
49             return sorted(gts)
50
51
52     def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
54         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
55
56         def predict_fn(cfg: Config) -> List[Interval]:
57             vt, mg, mwd = cfg
58             wins = TrackNetRunner.trajectory_to_action_windows(
59                 points, fps=fps, frame_width=frame_width,
60                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
61             )
62             return [(w["start"], w["end"]) for w in wins]
63
64         return predict_fn
65
66
67     def default_grid() -> List[Config]:
68         return [(vt, mg, mwd)
69                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
70                 for mg in (1.0, 2.0, 3.0)
71                 for mwd in (1.0, 2.0, 3.0)]
72
73
74     def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
75             iou: float = 0.5) -> dict:
76         gts = load_golden_gts(labels_csv)
77         if not gts:
78             raise SystemExit(f"no usable golden rallies in {labels_csv}")
79         predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
80         grid = default_grid()

```

```

81
82     base = evaluate(predict_fn(BASELINE), gts, iou)
83     best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
84     fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)    # OPTIMISTIC
(fit on full GT)
85     cv = cross_validate(predict_fn, grid, gts, k=5)                        # HONEST held-
out recall
86
87     print("=== WASB rally-segmentation calibration ===")
88     print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
89     print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
90           f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
91     print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
92     print(f" precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
93           f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
94     print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
95           f"({fit['f1_delta_pct']:+.1f}%)")
96     print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
97           f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
98           f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
99     print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
100     return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
101
102
103     def main():
104         ap = argparse.ArgumentParser()
105         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
106         ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
107         ap.add_argument("--fps", type=float, default=59.94)
108         ap.add_argument("--frame-width", type=float, default=1920.0)
109         ap.add_argument("--iou", type=float, default=0.5)
110         args = ap.parse_args()
111         run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
112
113
114     if __name__ == "__main__":
115         main()
116

```

4. user (2026-06-08T08:39:04.750Z)

```

1     """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2     on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3     100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5     Pipeline being tuned (all local, no Gemini at runtime):
6         WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7         -> rally_gate (min_crossings)                -> rally windows
8
9     Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10    videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11    the honest "how precise is the local pipeline on an unseen video" number. The
12    output is a single local config + the precision it buys, with NO Gemini dependency
13    shipped.
14
15    Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16    calibration.{select_best, leave_one_video_out, improvement_report}.
17
18    Run:

```

```

19     python -m backend.eval.calibrate_local --manifest videos.json
20     where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21     (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Callable, Dict, List, Tuple
28
29     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30     from backend.pipeline.detectors import rally_gate
31     from backend.eval.calibrate_wasb import load_golden_gts
32     from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34     # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35     LocalConfig = Tuple[float, float, float, int]
36     BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0) # today's local default (windowing,
no gate)
37
38
39     def make_local_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], list]:
40         """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
41         (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
42         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
43
44         def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:
45             vt, mg, mwd, mc = cfg
46             wins = TrackNetRunner.trajectory_to_action_windows(
47                 points, fps=fps, frame_width=frame_width,
48                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
49             )
50             wins = [(w["start"], w["end"]) for w in wins]
51             if mc > 0:
52                 wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
53             return wins
54
55         return predict_fn
56
57
58     def local_grid() -> List[LocalConfig]:
59         return [(vt, mg, mwd, mc)
60                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
61                 for mg in (1.0, 2.0, 3.0)
62                 for mwd in (1.0, 2.0, 3.0)
63                 for mc in (0, 1, 2, 3)]
64
65
66     def build_videos(specs: List[Dict]) -> List[Dict]:
67         videos = []
68         for s in specs:
69             pf = make_local_predict_fn(s["trajectory"], float(s["fps"]),
float(s["frame_width"]))
70             gts = load_golden_gts(s["labels"])
71             if not gts:
72                 raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
73             videos.append({"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts":
gts})
74         return videos
75
76
77     def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
78         videos = build_videos(specs)

```

```

79     grid = local_grid()
80     print(f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
81           f"{len(grid)} configs, IoU>={iou}, optimise {metric}) ===")
82
83     # Per-video: today's baseline vs the best-fit local config (optimistic ? fit on that
video).
84     per_video = []
85     for v in videos:
86         best_cfg, best_val = select_best(v["predict_fn"], grid, v["gts"], metric, iou)
87         imp = improvement_report(v["predict_fn"], BASELINE_LOCAL, best_cfg, v["gts"],
iou)
88         b, t = imp["baseline"], imp["tuned"]
89         print(f"\n[{v['name']}] {len(v['gts'])} Gemini rallies")
90         print(f"  baseline {BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f}
F1={b['f1']:.3f}")
91         print(f"  best-fit {best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f}
F1={t['f1']:.3f} "
92               f"(F1 {imp['f1_delta_pct']:+.0f}%, fit-on-self = optimistic)")
93         per_video.append({"name": v["name"], "best_cfg": best_cfg, "improvement": imp})
94
95     # Honest cross-video: pick config on the OTHER videos, score the held-out one.
96     lovo = leave_one_video_out(videos, grid, metric, iou)
97     # Baseline's held-out score (no tuning) for the honest before/after.
98     base_held = [evaluate(v["predict_fn"](BASELINE_LOCAL), v["gts"], iou) for v in
videos]
99     base_mean = sum(getattr(s, metric) for s in base_held) / len(base_held)
100
101     print("\n--- HONEST leave-one-video-out (the number to trust) ---")
102     if "error" in lovo:
103         print(f"  {lovo['error']} (need >=2 videos)")
104     else:
105         print(f"  baseline (untuned) mean held-out {metric}: {base_mean:.3f}")
106         print(f"  calibrated mean held-out {metric}:          {lovo['mean_held_out']:.3f}
"
107               f"± {lovo['std_held_out']:.3f} (gap {lovo['generalization_gap']:+.3f}")
108         for p in lovo["per_video"]:
109             print(f"    held-out {p['video']}: {p['held_out']:.3f} (cfg picked on
others: {p['config']})")
110     return {"per_video": per_video, "lovo": lovo, "baseline_held_out": base_mean,
"metric": metric}
111
112
113     def main() -> None:
114         ap = argparse.ArgumentParser(description="Calibrate the local-only pipeline against
Gemini silver labels.")
115         ap.add_argument("--manifest", required=True,
116                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
117         ap.add_argument("--iou", type=float, default=0.5)
118         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
119         args = ap.parse_args()
120         with open(args.manifest, encoding="utf-8") as f:
121             specs = json.load(f)
122         run(specs, iou=args.iou, metric=args.metric)
123
124
125     if __name__ == "__main__":
126         main()
127

```

5. user (2026-06-08T08:39:04.998Z)

```

1     """Evaluate the rally pipeline against PARTIAL human golden labels.
2
3     The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4     ending_reason,sport). While a video is only partially tagged, the labels cover a

```

```

5 PREFIX of the video ? every rally up to some point is marked, and everything after is
6 simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7 the detector's *correct* rallies in the un-labeled tail would be counted as false
8 positives and crush precision.
9
10 This harness fixes that by restricting the evaluation to the LABELED WINDOW
11 [window_start, window_end] (window_end defaults to the last labeled rally's end) and
12 dropping predicted segments that fall outside it. Within that window the human labels
13 ARE complete, so precision/recall/F1 are honest.
14
15 It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16 - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17 - pipeline.detectors.rally_gate.apply_gate (net-crossing "gate A")
18 - eval.metrics.match_intervals / score
19 - export_wasb_segments.build_comparison / write_comparison_csv (per-rally detail CSV)
20
21 Run:
22 python -m backend.eval.eval_partial_labels \
23     --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
24     --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
25     --fps 59.94 --frame-width 1920
26     ""
27 from __future__ import annotations
28
29 import argparse
30 import os.path as osp
31 from typing import List, Optional, Tuple
32
33 from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
34 from backend.eval.metrics import score
35 from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
36 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37 from backend.pipeline.detectors import rally_gate
38
39 Interval = Tuple[float, float]
40 Config = Tuple[float, float, float]
41
42 # Operating points to compare. The tuned config wins on its fit-video but over-segments
43 # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44 TUNED: Config = (0.05, 1.0, 1.0)
45
46
47 def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
48 List[Interval]:
49     """Keep intervals that overlap the labeled window [w0, w1].
50
51     A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
52     in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
53     they are neither rewarded nor penalized. GTs are assumed already inside the window.
54     """
55     return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
56
57 def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                 window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59     """Score one (config, gate) operating point inside the labeled window."""
60     preds = predict_fn(cfg)
61     if gate_crossings > 0 and points:
62         preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63     preds = restrict_to_window(preds, *window)
64     sc = score(preds, gts, iou)
65     return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
66 "preds": preds}

```

```

67
68 def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
69         window_start: float = 0.0, window_end: Optional[float] = None,
70         gate_crossings: int = 2, iou: float = 0.5,
71         comparison_out: Optional[str] = None) -> dict:
72     gts_all = load_golden_gts(labels_csv)
73     if not gts_all:
74         raise SystemExit(f"no usable human rallies in {labels_csv}")
75     if window_end is None:
76         window_end = max(e for _, e in gts_all)
77     window = (window_start, window_end)
78     gts = restrict_to_window(gts_all, *window)
79
80     predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
81     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
82     traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83
84     print("=== Pipeline vs PARTIAL human labels ===")
85     print(f"labels: {labels_csv}")
86     print(f" {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
{window_end:.1f}s] "
87           f"-> {len(gts)} rallies in-window")
88     print(f"trajectory covers ~0-{traj_end:.1f}s "
89           f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
than window'})")
90     print(f"IoU>={iou}; net-crossing gate K={gate_crossings}\n")
91
92     # Compare the standard operating points: baseline / baseline+gate / tuned /
tuned+gate.
93     points_for_gate = points
94     candidates = [
95         ("baseline", BASELINE, 0),
96         (f"baseline+gate{gate_crossings}", BASELINE, gate_crossings),
97         ("tuned", TUNED, 0),
98         (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
99     ]
100     results = []
101     header = f"{'operating point':<18} {'pred':>4} {'TP':>3} {'FN':>3} {'FP':>3} " \
102             f"{'prec':>6} {'recall':>6} {'F1':>6} {'mIoU':>6} {'sErr':>6} {'eErr':>6}"
103     print(header)
104     print("-" * len(header))
105     for name, cfg, gate in candidates:
106         r = _eval_point(predict_fn, cfg, points_for_gate, fps, gts, window, gate, iou)
107         sc = r["score"]
108         r["name"] = name
109         results.append(r)
110         print(f"{name:<18} {r['n_pred']:>4} {sc.true_positives:>3}
{sc.false_negatives:>3} "
111               f"{sc.false_positives:>3} {sc.precision:>6.3f} {sc.recall:>6.3f}
{sc.f1:>6.3f} "
112               f"{sc.mean_iou:>6.3f} {sc.mean_start_error:>6.2f}
{sc.mean_end_error:>6.2f}")
113
114     best = max(results, key=lambda r: r["score"].f1)
115     print(f"\nbest operating point by F1: {best['name']} (F1 {best['score'].f1:.3f})")
116
117     # Per-rally detail for the best point: which human rallies matched/missed + extra
fires.
118     comp = build_comparison(best["preds"], gts, iou)
119     comp_out = comparison_out or osp.join("output", "adarsh_avi_vs_human.csv")
120     write_comparison_csv(comp_out, comp)
121     print(f"per-rally detail ({best['name']}) -> {comp_out}")
122
123     return {
124         "window": window, "n_human_in_window": len(gts),

```

```

125         "results": [{"name": r["name"], "cfg": r["cfg"], "gate": r["gate"],
126                       "n_pred": r["n_pred"], **r["score"].as_dict()} for r in results],
127         "best": best["name"], "comparison_csv": comp_out,
128     }
129
130
131     def main() -> None:
132         ap = argparse.ArgumentParser(description=__doc__)
133         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
134         ap.add_argument("--labels", required=True, help="human golden CSV
(rally_number,start_time,end_time,...)")
135         ap.add_argument("--fps", type=float, default=59.94)
136         ap.add_argument("--frame-width", type=float, default=1920.0)
137         ap.add_argument("--window-start", type=float, default=0.0)
138         ap.add_argument("--window-end", type=float, default=None,
139                         help="end of the labeled region (default: last labeled rally end)")
140         ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
recommended)")
141         ap.add_argument("--iou", type=float, default=0.5)
142         ap.add_argument("--comparison-out", default=None)
143         args = ap.parse_args()
144         run(args.trajectory, args.labels, args.fps, args.frame_width,
145             window_start=args.window_start, window_end=args.window_end,
146             gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
147
148
149     if __name__ == "__main__":
150         main()
151

```

6. user (2026-06-08T08:39:05.173Z)

```

1     """Slice individual rally clips out of a (long) annotated video.
2
3     Use cases:
4     - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5       boundaries against the labels.
6     - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7       supervision the segmenter is tuned/adapted on for that specific video.
8
9     Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10      end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11      <out_dir>/<video_stem>__rally_<n>.mp4
12
13     Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14     shows the same lead-in/out the model is trained and evaluated against. The window
15     math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16     `slice_rallies`.
17     """
18     from __future__ import annotations
19
20     import argparse
21     import csv
22     import os
23     import subprocess
24     from dataclasses import dataclass
25     from typing import Dict, List, Optional, Tuple
26
27     from backend.eval.annotations import parse_timestamp
28
29     # Match VideoCompiler defaults so sliced clips frame a rally the same way.
30     BEGIN_PADDING = 0.5
31     END_PADDING = 1.0
32

```



```

33
34 @dataclass
35 class ClipWindow:
36     rally: str          # kept as a string to allow inserted ids like "7.5"
37     start: float        # padded, clamped
38     end: float          # padded, clamped
39
40
41 def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
42     """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43     for key in (decimal_key, mmss_key):
44         val = (row.get(key) or "").strip()
45         if val:
46             try:
47                 return parse_timestamp(val)
48             except ValueError:
49                 continue
50     return None
51
52
53 def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54     """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56     Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57     end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58     """
59     labels: Dict[str, Tuple[float, float]] = {}
60     with open(csv_path, newline="", encoding="utf-8") as f:
61         reader = csv.DictReader(f)
62         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63 [])]
64         for row in reader:
65             row = {(k or "").strip().lower(): v for k, v in row.items()}
66             rally = (row.get("rally_number") or "").strip()
67             if not rally:
68                 continue
69             start = _read_time(row, "start_time", "start_mmss")
70             end = _read_time(row, "end_time", "end_mmss")
71             if start is None or end is None or end <= start:
72                 continue # missing/degenerate (e.g. a not-yet-filled MISSED row)
73             labels[rally] = (start, end)
74     return labels
75
76
77 def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
78 rally_numbers: List[str],
79 begin_pad: float = BEGIN_PADDING,
80 end_pad: float = END_PADDING,
81 video_duration: Optional[float] = None) -> List[ClipWindow]:
82     """Apply padding and clamp to the video, for the requested rallies.
83
84     rally_numbers may be ["all"] to take every labeled rally (in time order).
85     Unknown rally ids are skipped (caller can diff to warn).
86     """
87     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
88         wanted = sorted(labels, key=lambda r: labels[r][0])
89     else:
90         wanted = [r for r in rally_numbers if r in labels]
91
92     windows: List[ClipWindow] = []
93     for rally in wanted:
94         start, end = labels[rally]
95         ps = max(0.0, start - begin_pad)
96         pe = end + end_pad
97         if video_duration is not None and video_duration > 0:

```

```

97         if ps >= video_duration:
98             continue # entirely past the end
99         pe = min(pe, video_duration)
100     if pe <= ps:
101         continue
102     windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103 return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(
109             ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111             capture_output=True, text=True, check=True,
112             ).stdout.strip()
113         return float(out)
114     except (subprocess.SubprocessError, OSError, ValueError):
115         return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []
136     for w in windows:
137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,
145         ]
146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151             continue
152         if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153             written.append(out_path)
154             print(f"[slicer] rally {w.rally} -> {out_path}")
155     return written
156
157
158 def main() -> None:
159     p = argparse.ArgumentParser(description="Slice rally clips from an annotated
160 video.")

```

```

160     p.add_argument("--video", required=True, help="Source video file")
161     p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
162     p.add_argument("--rallies", required=True,
163                    help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
164     p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
165     p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
166     p.add_argument("--end-pad", type=float, default=END_PADDING)
167     args = p.parse_args()
168
169     rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
170     written = slice_rallies(args.video, args.labels, rally_numbers,
171                            args.out_dir, args.begin_pad, args.end_pad)
172     print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
173
174
175     if __name__ == "__main__":
176         main()
177

```

7. user (2026-06-08T08:39:05.349Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10    filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11    active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12    -> rally windows.
13
14    Training labels come from Gemini full-context labels via the SAME IoU matching the
15    evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16    other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17    compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18    rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20    Run:
21    python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22    """
23    from __future__ import annotations
24
25    import json
26    import argparse
27    from typing import Dict, List, Tuple
28
29    import numpy as np
30
31    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
32    from backend.pipeline.detectors import rally_gate
33    from backend.eval.calibrate_wasb import load_golden_gts
34    from backend.eval.training import label_candidates
35    from backend.eval.classifier import LogisticRegression
36    from backend.eval.metrics import score
37    from backend.eval.gemini_refine import merge_overlaps
38
39    Interval = Tuple[float, float]
40
41    FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42    # Recall-oriented proposal: deliberately permissive so real rallies are among the

```

```

43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44     PROPOSAL = (0.005, 1.0, 0.5)
45     BASELINE_LOCAL = (0.02, 2.0, 2.0)    # today's production-ish windowing, no learning
46
47
48     def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
49                          proposal: Tuple[float, float, float] = PROPOSAL) ->
50     Tuple[List[Interval], List[List[float]]]:
51         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
52         rows)."""
53         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
54         vt, mg, mwd = proposal
55         wins = TrackNetRunner.trajectory_to_action_windows(
56             points, fps=fps, frame_width=frame_width,
57             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
58         intervals = [(w["start"], w["end"]) for w in wins]
59         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
60         crossings
61         X: List[List[float]] = []
62         for w, g in zip(wins, gated):
63             dur = max(1e-6, w["end"] - w["start"])
64             win_frames = max(1.0, dur * fps)
65             X.append([
66                 dur,                                # rally length (sec) ?
67                 float(w["peak_velocity"]),           # already normalized by
68                 float(w["mean_velocity"]),           # active-shuttle ratio ?
69                 float(w["active_frame_count"]) / win_frames,
70                 float(g.crossings),                  # net crossings (count) ?
71                 the rally signal
72             ])
73         return intervals, X
74
75     def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
76     List[Interval]:
77         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
78         vt, mg, mwd = BASELINE_LOCAL
79         wins = TrackNetRunner.trajectory_to_action_windows(
80             points, fps=fps, frame_width=frame_width,
81             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82         return [(w["start"], w["end"]) for w in wins]
83
84     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
85         fps, fw = float(spec["fps"]), float(spec["frame_width"])
86         intervals, X = build_candidates(spec["trajectory"], fps, fw)
87         gts = load_golden_gts(spec["labels"])
88         y = label_candidates(intervals, gts, iou)
89         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
90         X,
91                 "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
92         fw)}
93
94     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
95     List[Interval],
96         threshold: float = 0.5) -> List[Interval]:
97         if not intervals:
98             return []
99         proba = model.predict_proba(X)
100         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
101         return merge_overlaps(pos, gap_tol=1.0)

```

```

97
98
99     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5, model_out: str =
None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
111             clf, base = [], []
112             for i, held in enumerate(videos):
113                 others = [v for j, v in enumerate(videos) if j != i]
114                 X = [row for v in others for row in v["X"]]
115                 y = [t for v in others for t in v["y"]]
116                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118                 sc = score(preds, held["gts"], iou)
119                 bsc = score(held["baseline"], held["gts"], iou)
120                 clf.append(sc)
121                 base.append(bsc)
122                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}")
123                 f"    | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124             mean = lambda xs, a: sum(getattr(s, a) for s in xs) / len(xs)
125             print(f"  MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
126                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
127                   f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
128             result["mean_learned_f1"] = mean(clf, "f1")
129             result["mean_baseline_f1"] = mean(base, "f1")
130
131             # Final model on ALL videos (for production / running on a fresh held-out video).
132             Xall = [row for v in videos for row in v["X"]]
133             yall = [t for v in videos for t in v["y"]]
134             final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
135             weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
136             print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
137             if model_out:
138                 final.save(model_out)
139                 print(f"final model (all videos) -> {model_out}")
140                 result["model_out"] = model_out
141             return result
142
143
144     def main() -> None:
145         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
146         ap.add_argument("--manifest", required=True,
147                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
148         ap.add_argument("--iou", type=float, default=0.5)
149         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
150         ap.add_argument("--model-out", default="output/local_rally_model.json")
151         args = ap.parse_args()
152         with open(args.manifest, encoding="utf-8") as f:
153             specs = json.load(f)

```

```

154         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
155
156
157     if __name__ == "__main__":
158         main()
159

```

8. assistant (2026-06-08T08:39:07.804Z)

Now let me check the training.py file to understand label_candidates and how labels feed into the training pipeline:

9. user (2026-06-08T08:39:08.441Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/training.py

10. user (2026-06-08T08:39:10.263Z)

```

1      """Build a training set for the offline, API-free learned rally segmenter.
2
3      Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4      a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5      produces candidate rally windows with features; the collector's ground-truth
6      rally intervals tell us which windows are real. A lightweight classifier learns
7      that mapping, replacing the paid Gemini refinement with a local model.
8
9      This module is the *data layer* ? pure and unit-tested:
10
11      - `label_candidates` ? generic: label windows positive/negative by whether they
12        match a ground-truth rally (same IoU matching the evaluator uses, so training
13        labels are consistent with how we score).
14      - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15        persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16      - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18      No model training or I/O here; that lives in the trainer/segmenter once the
19      substrate is chosen from the Phase 3 candidate-recall result.
20      """
21
22      from typing import Callable, Dict, List, Tuple
23
24      from backend.eval.metrics import Interval, match_intervals
25
26      # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27      # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28      YOLO_FEATURE_NAMES = [
29          "duration", "peak_velocity", "mean_velocity", "active_frame_count",
30          "track_id_count",
31      ]
32
33      def extract_yolo_features(row: Dict) -> List[float]:
34          """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36          `stats` is already JSON-deserialized by the DB layer. Missing fields default
37          to 0.0 so a sparse/old row still yields a well-formed vector.
38          """
39          stats = row.get("stats") or {}
40          duration = float(row["end_time"]) - float(row["start_time"])
41          return [
42              duration,
43              float(stats.get("peak_velocity", 0.0)),
44              float(stats.get("mean_velocity", 0.0)),
45              float(stats.get("active_frame_count", 0)),

```

```

46         float(stats.get("track_id_count", 0)),
47     ]
48
49
50 def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51                     iou_threshold: float = 0.5) -> List[int]:
52     """Label each candidate window 1 (real rally) or 0 (spurious).
53
54     A window is positive iff it is matched one-to-one to a ground-truth rally at
55     `iou_threshold` ? the *same* matching the evaluator uses for true positives,
56     so the classifier is trained on exactly what we score. Substrate-agnostic:
57     works for YOLO or WASB-derived windows alike.
58     """
59     mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
60     positive = {m.pred_index for m in mr.matches}
61     return [1 if i in positive else 0 for i in range(len(candidate_intervals))]
62
63
64 def build_training_set(
65     candidate_rows: List[Dict],
66     gt_intervals: List[Interval],
67     iou_threshold: float = 0.5,
68     feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
69 ) -> Tuple[List[List[float]], List[int], List[Interval]]:
70     """Turn one video's candidate rows + ground truth into (X, y, intervals).
71
72     `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
73     features without changing the labeling logic. Returns the feature matrix `X`,
74     binary labels `y`, and the candidate intervals (handy for downstream merging).
75     """
76     intervals: List[Interval] = [
77         (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
78     ]
79     y = label_candidates(intervals, gt_intervals, iou_threshold)
80     X = [feature_fn(r) for r in candidate_rows]
81     return X, y, intervals
82

```

11. assistant (2026-06-08T08:39:12.560Z)

Now let me check the metrics.py to understand IoU matching and the Interval type:

12. user (2026-06-08T08:39:13.405Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/metrics.py

13. user (2026-06-08T08:39:15.353Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.

```

```

17
18     Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19     overlap / union, both measured as durations on the timeline.
20     """
21     a_start, a_end = a
22     b_start, b_end = b
23     if a_end <= a_start or b_end <= b_start:
24         return 0.0
25     inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26     if inter <= 0.0:
27         return 0.0
28     union = (a_end - a_start) + (b_end - b_start) - inter
29     return inter / union if union > 0 else 0.0
30
31
32 @dataclass
33 class Match:
34     """One matched predicted?ground-truth pair."""
35     pred_index: int
36     gt_index: int
37     iou: float
38
39
40 @dataclass
41 class MatchResult:
42     """Outcome of matching predictions to ground truth at a given IoU threshold."""
43     matches: List[Match] = field(default_factory=list)
44     unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45     unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
46 (misses)
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49                    iou_threshold: float = 0.5) -> MatchResult:
50     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             if iou >= iou_threshold:
62                 candidates.append((iou, pi, gi))
63     # Highest IoU first; ties broken by indices for determinism.
64     candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
65
66     used_pred, used_gt = set(), set()
67     matches: List[Match] = []
68     for iou, pi, gi in candidates:
69         if pi in used_pred or gi in used_gt:
70             continue
71         used_pred.add(pi)
72         used_gt.add(gi)
73         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
74
75     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
76     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
77     return MatchResult(matches=matches,
78                        unmatched_pred_indices=unmatched_pred,
79                        unmatched_gt_indices=unmatched_gt)
80

```



```

81
82 @dataclass
83 class Score:
84     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
85     iou_threshold: float
86     num_pred: int
87     num_gt: int
88     true_positives: int
89     false_positives: int
90     false_negatives: int
91     precision: float
92     recall: float
93     f1: float
94     mean_iou: float          # over matched pairs (0.0 if none)
95     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
96     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
97
98     def as_dict(self) -> dict:
99         return self.__dict__.copy()
100
101
102 def score(preds: List[Interval], gts: List[Interval],
103          iou_threshold: float = 0.5,
104          match_result: Optional[MatchResult] = None) -> Score:
105     """Compute precision/recall/F1, mean matched IoU, and boundary errors.
106
107     Pass a precomputed `match_result` to avoid re-matching (e.g. when also
108     extracting the failure lists); otherwise matching is done here.
109     """
110     mr = match_result if match_result is not None else match_intervals(preds, gts,
111 iou_threshold)
112
112     tp = len(mr.matches)
113     fp = len(mr.unmatched_pred_indices)
114     fn = len(mr.unmatched_gt_indices)
115
116     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
117     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
118     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
119 else 0.0
120
120     if mr.matches:
121         mean_iou = sum(m.iou for m in mr.matches) / tp
122         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
123 mr.matches) / tp
124         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
125 mr.matches) / tp
126     else:
127         mean_iou = mean_start_err = mean_end_err = 0.0
128
129     return Score(
130         iou_threshold=iou_threshold,
131         num_pred=len(preds), num_gt=len(gts),
132         true_positives=tp, false_positives=fp, false_negatives=fn,
133         precision=precision, recall=recall, f1=f1,
134         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
135     )
136
137
138 def pr_curve(preds: List[Interval], gts: List[Interval],
139             thresholds: Optional[List[float]] = None) -> List[Score]:
140     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142     Lets you see strict-vs-lenient behaviour rather than a single operating point.
143     """

```

```
142         if thresholds is None:
143             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
144         return [score(preds, gts, iou_threshold=t) for t in thresholds]
145
```

14. assistant (2026-06-08T08:39:17.710Z)

Now let me check for any CSV schemas or documentation, and also look at how the collector might output data:

15. user (2026-06-08T08:39:18.333Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.pytest_cache/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/checkpoints/2026-06-eval-
baseline/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/checkpoints/2026-06-logging-
reporting/README.md
C:\Users\avidu\Projects\badminton-highlight-
indexer/docs/archives/checkpoints/2026-06-scaffolding-complete/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/COMMERCIAL_READINESS_REVIEW.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/MONETIZATION_AUDIT.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/low-regret-
moves/LOW_REGRET_MOVES_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/past_projects/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/quality-
iterations/2026-06-initial/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/quality-iterations/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/archives/research/AUDIO_E1_FINDINGS.md
C:\Users\avidu\Projects\badminton-highlight-
indexer/docs/archives/research/AUDIO_RALLY_DETECTION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-
indexer/docs/archives/research/RALLY_DETECTION_RESEARCH.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/BACKLOG.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/CODE_MAP.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/COMMERCIALIZATION.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/DEFERRED_HARDENING_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/EVALUATION.md
```

16. user (2026-06-08T08:39:20.210Z)

```
1      # Annotation & Evaluation
2
3      How to measure how well the pipeline detects rallies, using hand-annotated
4      ground-truth timestamps. The harness scores predictions **already stored in the
5      SQLite DB** ? it makes no API calls and is fully deterministic, so you can run it
6      as often as you like for free.
7
8      ---
9
10     ## 1. What you annotate (and what it does / doesn't do)
11
12     You annotate, per video, the `[start, end)` time range of each **rally**. That's
13     all the evaluation harness needs.
14
15     > **Important:** these rally timestamps measure and tune **rally segmentation**
16     > ? i.e. *where* the pipeline says rallies begin and end. They do **not** train
17     > the shuttle detector (TrackNet/WASB), which learns from per-frame shuttle
18     > `(x, y)` coordinate labels, a different and much heavier annotation. Use rally
19     > timestamps as your **golden evaluation set**; only invest in frame-level
20     > coordinate labels if evaluation shows *detection* (not segmentation) is the
21     > bottleneck. See `docs/TRACKNET_WSL_SETUP.md`.
22
```

```

23  ## 2. Annotation file format (CSV)
24
25  One CSV per video, one rally per row:
26
27  ```csv
28  start,end
29  00:01:12,00:01:39
30  102.5,131.0
31  ```
32
33  - The `start,end` header is optional (auto-detected).
34  - Timestamps may be **decimal seconds** (`102.5`) or **colon-separated**
35    `MM:SS` / `HH:MM:SS` (`00:01:12`). Forms can be mixed.
36  - Blank lines and `#` comments are ignored.
37  - Malformed rows (bad timestamp, `start >= end`) fail loudly rather than being
38    silently skipped ? so labeling mistakes surface immediately.
39
40  Generate an empty template to fill in:
41
42  ```bash
43  python run_eval.py --scaffold --annotations rallies.csv
44  ```
45
46  ## 3. Running the evaluation
47
48  First make sure the video has been processed so its detections are in the DB
49  (the pipeline keys each video by its file MD5). Then:
50
51  ```bash
52  # Identify the video by file (its MD5 is computed for you)...
53  python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv
54
55  # ...or by the stored video_id (MD5) directly:
56  python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
57  ```
58
59  Useful flags:
60
61  | Flag | Effect |
62  |-----|-----|
63  | `--stage {candidates,final,both}` | Which prediction stage(s) to score (default
64  `both`). |
65  | `--detector <name>` | Only score candidates from one detector (e.g. `yolo_hybrid`,
66  `tracknet_hybrid`). |
67  | `--iou <float>` | Match threshold (default `0.5`). |
68  | `--pr-curve` | Also report an IoU sweep (0.1?0.9). |
69  | `--show-failures` | List missed (FN) and spurious (FP) rallies with timestamps. |
70  | `--json <path>` | Write a machine-readable report. |
71  | `--db <path>` | Override the DB path (default: `output/<output.db_name>` from
72  `config.json`). |
73
74  ## 4. Reading the output
75
76  ```
77  =====
78  Rally evaluation ? video 'demo'
79  Ground-truth rallies: 3 | IoU threshold: 0.5
80  =====
81  Stage      Pred  TP  FP  FN  Prec  Rec  F1  mIoU  dStart  dEnd
82  -----
83  candidates    3   3   0   0  1.00  1.00  1.00  1.00  0.0s   0.0s
84  final         2   2   0   1  1.00  0.67  0.80  0.95  0.2s   0.5s
85  =====
86  [final] missed rallies (false negatives): 1
87  - 00:01:20?00:01:32

```

```

85     ``
86
87     - **TP / FP / FN** ? true positives (matched), false positives (spurious
88       predictions), false negatives (missed rallies).
89     - **Prec / Rec / F1** ? precision, recall, and their harmonic mean.
90     - **mIoU** ? mean temporal IoU over matched pairs (how tightly matched rallies
91       overlap their ground truth).
92     - **dStart / dEnd** ? mean absolute boundary error in seconds (are rallies
93       detected but trimmed in the wrong place?).
94
95     ### The key diagnostic: candidates vs. final
96
97     Scoring both stages tells you where to spend effort:
98
99     - **Candidates miss rallies the GT has** ? the **detector** is the bottleneck.
100       Improve the shuttle model (e.g. fine-tune TrackNet/WASB with frame-level
101       labels) or relax detection thresholds.
102     - **Candidates catch them but `final` is wrong/trimmed** ? the **segmentation /
103       AI handoff** is the bottleneck. Tune thresholds (`velocity_threshold`,
104       `dead_time_merge_gap`, `min_action_window_duration`) or the AI prompt ? no new
105       labels needed.
106
107     In the example above, detection is perfect (candidates: recall 1.0) but the AI
108     handoff dropped one rally (final: recall 0.67) ? so the fix is in segmentation,
109     not detection.
110
111     ## 5. How matching works
112
113     Predictions and ground truth are sets of `[start, end)` intervals. A prediction
114     **matches** a ground-truth rally when their **temporal IoU** ( $\text{overlap} \div \text{union}$ ) is
115     at least the threshold. Matching is **greedy and one-to-one** ? highest-IoU pairs
116     are claimed first, and each prediction/GT is used at most once, so a single
117     detection can't be credited for two rallies. Unmatched predictions are false
118     positives; unmatched ground-truth rallies are false negatives.
119
120     ## 6. Code map
121
122     | File | Role |
123     |-----|-----|
124     | `backend/eval/metrics.py` | Pure interval math: IoU, matching, P/R/F1, boundary error.
125     |
126     | `backend/eval/annotations.py` | Load/validate the ground-truth CSV. |
127     | `backend/eval/harness.py` | Read predictions from the DB, score, format the report. |
128     | `run_eval.py` | CLI entry point. |
129     | `tests/test_eval_*.py` | Deterministic tests for all of the above. |
130
131     ## 7. Troubleshooting
132
133     | Symptom | Cause & fix |
134     |-----|-----|
135     | `ERROR: database not found at output/...` | The harness only reads stored predictions
136       ? it does **not** run the pipeline. **Process the video first** (e.g.
137       `run_process_and_stitch.py`) so detections land in the DB, then re-run. |
138     | Every rally shows as a miss (FN), `Pred` is 0 | This video hasn't been processed, or
139       you used the wrong `video_id`. The pipeline keys videos by **file MD5** ? pass the same file
140       via `--video`, or the exact stored `--video-id`. |
141     | `video_id` doesn't match although it's "the same" video | Re-encoding/trimming changes
142       the bytes, hence the MD5. Use the exact file that was processed, or look up the stored id (it's
143       printed during processing). |
144     | `candidates` stage is empty but `final` has data (or vice-versa) | Depends on which
145       segmenter ran. `gemini` writes only `final` (no candidates); `yolo_hybrid`/`tracknet_hybrid`
146       write both. Use `--stage final` for the gemini segmenter, and `--detector <name>` to scope
147       candidates. |
148     | Recall looks low only at high `--iou` | Rallies are found but boundaries are loose ?
149       check `dStart`/`dEnd`. Tune `stitching` padding or the segmentation thresholds rather than the

```

```

detector. |
139 | `ValueError: start (...) >= end (...)` while loading annotations | A bad row in your
CSV. Timestamps must be increasing per rally and use seconds or `HH:MM:SS`. |
140
141 Run `python run_eval.py -h` for a quick reminder of this prerequisite and all flags.
142

```

17. assistant (2026-06-08T08:39:22.667Z)

Now let me check the annotations.py to see all supported CSV column names:

18. user (2026-06-08T08:39:23.427Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/annotations.py

19. user (2026-06-08T08:39:25.332Z)

```

1      """Load hand-annotated ground-truth rally timestamps from CSV.
2
3      Annotation file format (one file per video):
4
5          start,end
6          00:01:12,00:01:39
7          102.5,131.0
8
9      - A header row `start,end` is optional (auto-detected and skipped).
10     - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11       `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12     - Blank lines and `#` comment lines are ignored.
13     """
14
15     import csv
16     import logging
17     from typing import List, Tuple
18
19     logger = logging.getLogger(__name__)
20
21     Interval = Tuple[float, float]
22
23
24     def parse_timestamp(value: str) -> float:
25         """Parse a timestamp string to float seconds.
26
27         Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28         Raises ValueError on anything unparseable so bad annotations fail loudly.
29         """
30         value = value.strip()
31         if not value:
32             raise ValueError("empty timestamp")
33         if ":" in value:
34             parts = value.split(":")
35             if len(parts) > 3:
36                 raise ValueError(f"too many ':' segments in timestamp '{value}'")
37             total = 0.0
38             for part in parts:                                # most-significant first (H, M, S)
39                 total = total * 60 + float(part)
40             return total
41         return float(value)
42
43
44     def load_annotations(csv_path: str) -> List[Interval]:
45         """Load and validate ground-truth rally intervals from a CSV file.
46
47         Returns intervals sorted by start time. Raises ValueError on malformed rows

```

```

48         (bad timestamp, start >= end) so labeling mistakes surface immediately rather
49         than silently skewing scores.
50         """
51         intervals: List[Interval] = []
52         with open(csv_path, newline="") as f:
53             reader = csv.reader(f)
54             for lineno, row in enumerate(reader, start=1):
55                 if not row:
56                     continue
57                 first = row[0].strip()
58                 if not first or first.startswith("#"):
59                     continue
60                 # Skip an optional header row.
61                 if first.lower() in ("start", "start_time"):
62                     continue
63                 if len(row) < 2:
64                     raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
65                 try:
66                     start = parse_timestamp(row[0])
67                     end = parse_timestamp(row[1])
68                 except ValueError as e:
69                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
70                 if start >= end:
71                     raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
72                 intervals.append((start, end))
73
74         intervals.sort(key=lambda iv: iv[0])
75         logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
76         return intervals
77
78
79     def write_scaffold(csv_path: str) -> None:
80         """Write an empty annotation file with just the header, for hand-filling."""
81         with open(csv_path, "w", newline="") as f:
82             writer = csv.writer(f)
83             writer.writerow(["start", "end"])
84

```

20. assistant (2026-06-08T08:39:27.692Z)

Now let me look at what the collector likely produces. Let me search for the collector project:

21. user (2026-06-08T08:39:27.910Z)

Exit code 2

```
/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''
```

22. user (2026-06-08T08:39:32.632Z)

Directory: C:\Users\avidu\Projects

Mode Name	LastWriteTime	Length
----	-----	-----

d----	6/3/2026 5:14 PM	.NET
d----	6/3/2026 7:51 PM	.NET Aspire
d----	6/6/2026 8:01 PM	Annotation Setup
d----	6/8/2026 7:11 AM	badminton-highlight-indexer
d----	5/30/2026 2:06 PM	badminton-highlight-indexer-
khelacharya		
d----	6/5/2026 7:06 PM	CODE WITH ERIC
d----	4/17/2026 7:51 PM	Email-Mining

d----	8/17/2023	7:24 PM	gtest
d----	6/4/2026	7:43 AM	muneem
d----	6/8/2026	1:56 PM	rally-annotator
d----	6/7/2026	2:21 PM	sports-data-collector
d----	6/7/2026	1:13 PM	sports-obsreport
d----	6/7/2026	11:41 PM	Verifiers
-a---	6/3/2026	10:57 PM	5773 antigravity_analysis_results.md
-a---	6/2/2026	1:05 PM	88321 KhelSutraGuru.jpeg

23. assistant (2026-06-08T08:39:35.274Z)

Let me check the Annotation Setup directory to see what format the collector produces:

24. user (2026-06-08T08:39:42.190Z)

Exit code 1

head: The term 'head' is not recognized as a name of a cmdlet, function, script file, or executable program.

Check the spelling of the name, or if a path was included, verify that the path is correct and try again.

25. user (2026-06-08T08:39:46.428Z)

Directory: C:\Users\avidu\Projects\Annotation Setup\Collect

Mode	LastWriteTime	Length
Name		
----	-----	-----

-a---	6/8/2026 1:48 PM	1751 Adarsh and Avi.rallies.csv

26. user (2026-06-08T08:39:48.390Z)

1	rally_number,start_time,end_time,ending_reason,sport
2	1,37.434,72.194,winner,badminton
3	2,79.936,112.954,winner,badminton
4	3,130.193,134.947,winner,badminton
5	4,139.697,144.944,unforced_error,badminton
6	5,152.196,157.198,unforced_error,badminton
7	6,161.694,165.950,unforced_error,badminton
8	7,170.817,176.327,forced_error,badminton
9	8,182.829,186.083,unforced_error,badminton
10	9,193.576,199.831,forced_error,badminton
11	10,205.832,211.828,unforced_error,badminton
12	11,221.082,225.827,forced_error,badminton
13	12,232.832,236.829,forced_error,badminton
14	13,241.828,246.329,forced_error,badminton
15	14,252.329,258.080,unforced_error,badminton
16	15,266.081,271.083,forced_error,badminton
17	16,276.577,283.578,forced_error,badminton
18	17,289.830,293.579,unforced_error,badminton
19	18,300.329,310.079,forced_error,badminton
20	19,316.829,319.585,unforced_error,badminton
21	20,324.576,328.324,unforced_error,badminton
22	21,332.581,336.829,unforced_error,badminton
23	22,345.577,348.830,unforced_error,badminton
24	23,356.591,359.826,unforced_error,badminton
25	24,365.578,376.823,unforced_error,badminton
26	25,383.075,384.580,service_fault,badminton
27	26,389.833,396.577,unforced_error,badminton
28	27,402.830,406.581,winner,badminton
29	28,414.578,431.828,winner,badminton
30	29,440.327,443.829,unforced_error,badminton
31	30,453.829,462.828,winner,badminton

```
32      31,470.829,477.079,unforced_error,badminton
33      32,487.329,497.573,winner,badminton
34      33,503.329,507.577,forced_error,badminton
35      34,519.077,524.579,winner,badminton
36      35,533.580,536.828,winner,badminton
37      36,543.829,547.079,unforced_error,badminton
38      37,554.706,557.964,unforced_error,badminton
39      38,566.462,578.716,unforced_error,badminton
40      39,586.467,589.714,unforced_error,badminton
41      40,599.712,604.962,unforced_error,badminton
42
```

27. assistant (2026-06-08T08:39:51.204Z)

Perfect! Now let me look for any documentation about what the collector outputs and check the sports-data-collector repo:

28. user (2026-06-08T08:39:51.670Z)

```
C:\Users\avidu\Projects\sports-data-collector/.pytest_cache/README.md
C:\Users\avidu\Projects\sports-data-collector\data/validation/pilot_badminton_01_report.md
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/NDA_TEMPLATE.md
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/ROORA_BRIEF.md
C:\Users\avidu\Projects\sports-data-collector/docs/DECISIONS.md
C:\Users\avidu\Projects\sports-data-collector/docs/INGESTION_PIPELINE.md
C:\Users\avidu\Projects\sports-data-collector/docs/PIPELINE_ROLE.md
C:\Users\avidu\Projects\sports-data-collector/README.md
```

29. user (2026-06-08T08:39:54.135Z)

```
1      # Role in the rally-segmenter training pipeline
2
3      This repo is the data producer in a two-repo system. Its job is to curate
4      sports videos with per-rally `[start, end)` annotations and commercial-license
5      metadata, then hand that data to the badminton-highlight-indexer, which
6      processes, scores, and (Phase 4) trains an offline rally segmenter on it.
7
8      This doc explains why the `export` and `fetch` commands exist and how they fit
9      the bigger picture. The full decision records and end-to-end narrative live in the
10     indexer repo:
11
12     - `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
13     - `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
14       (acquisition policy), ADR-004 (offline learned segmenter).
15
16     ## Why this repo matters
17
18     The indexer's best rally mode refines local detections with a paid Gemini API.
19     The strategic goal is to replace that with an offline model trained on our own
20     commercially-licensed, annotated corpus ? making rally detection free, offline,
21     and self-improving. None of that is possible without a clean, licensable,
22     accurately-labelled, full-resolution dataset. That dataset is what this repo
23     produces.
24
25     ## What the two key commands do (and why)
26
27     - curate export ? emits the indexer's ground-truth format: per-video
28       `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29       Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30       table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31       `is_commercial`, so only licensable data flows
32       downstream. The annotations are temporal-only ? they train/evaluate rally
33       segmentation, not the frame-level shuttle detector (see ADR-002).
```



```

34
35 - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36 the DB **in place** so annotations and curation status are never lost. Uses
37 authenticated cookies for reliable, unthrottled downloads. The why and the
38 hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40 ## The one contract that matters: video identity
41
42 The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43 re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44 exact `local_path`, and any downstream processing must run on **that** file. This is
45 why acquisition (`fetch`) happens *before* the indexer processes anything.
46

```

30. assistant (2026-06-08T08:39:56.124Z)

Now let me check the ingestion pipeline docs:

31. user (2026-06-08T08:39:56.203Z)

```

1 # Rally Ingestion Pipeline ? End-to-End FAQ
2
3 How a vendor's rally annotations become training/eval ground truth, and how we
4 validate each delivery against our conventions. This is the operational companion
5 to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7 ```
8     vendor delivery                OUR side (collector)                modeling
(indexer)
9     ?????????????????? convert ?????????????????? validate ??????????????????
10    ? CVAT export ? ???????????? ? canonical CSV ? ?????????? ? report .md ?
11    ? (boxes + attrs)? ? (per rally, ? ? report .json ?
12    ? OR ? ? ? decimal secs)? ? review .csv ??? human
verifies
13    ? canonical CSV ? ?????????????????? ??????????????????
flagged rallies
14    ?????????????????? ? ?
15    ? import-local (validated) ?
16    ? golden labels
17    ?????????????????? export ??????????????????
18    ? SQLite (rally ? ?????????? ? <id>.csv ?
start,end (indexer GT)
19    ? tables) ? ? manifest.json?
20    ?????????????????? ??????????????????
21    ?
22    slice clips / calibrate /
train
23    ```
24
25 ## The four commands (in order)
26
27 ```bash
28 # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)
29 python -m src.cli.curate convert-cvat --input job.xml \
30     --video-path match.mp4 --out canonical.csv --issues issues.txt
31
32 # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33 python -m src.cli.curate validate-delivery --input job.xml \
34     --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35 # ...or validate a CSV directly:
36 python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
pilot_badminton_01
37
38 # 3. After human review, import the (corrected) CSV into the DB

```

```

39 python -m src.cli.curate import-local --sport badminton \
40     --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42 # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43 python -m src.cli.curate export --out-dir data/eval_export
44 ```
45
46 ---
47
48 ## FAQ
49
50 ### What delivery formats do we accept?
51 Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
52 attributes) is normalised by `convert-cvat`; an already-canonical CSV is taken
53 as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
54 CSVs.
55
56 ### What is the "canonical" CSV?
57 One row per rally:
58 `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
59 last_shot_outcome, score_before, score_after, notes`.
60 `start_time`/`end_time` are decimal seconds; `import-local` reads those.
61
62 ### How are timestamps handled? (the fps thing)
63 We recompute each rally's interval from its frame range × the true fps
64 (`seconds = frame / fps`). A vendor's CVAT time attributes are ignored ? a
65 step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
66 from `--fps` or ffprobe of `--video-path`. `mm:ss` is also accepted everywhere.
67
68 ### What does validation check?
69 | code | severity | meaning |
70 |---|---|---|
71 | `overlap` | blocker | two rallies' intervals overlap (`end[N] > start[N+1]`) |
72 | `zero_duration` | blocker | `end <= start` (single-sample / stray box) |
73 | `frame_out_of_range` | high | a box frame is past the video's last frame |
74 | `shot_density` | medium | long rally with implausibly few shots ? likely an inflated
75 span (often hides a merged rally) |
76 | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
77 for an omitted rally |
78 | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
79 | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
80
81 ### Blocker vs. review ? what's the gate?
82 `validate-delivery` exits non-zero if any blocker exists (so it can gate a bulk
83 loop / CI). Non-blocker issues don't stop the gate but populate the review sheet
84 for the human pass. `import-local` independently rejects overlaps/zero-duration, so
85 bad ground truth can't slip into the DB.
86
87 ### How are missed rallies caught? (the worst error)
88 Two complementary checks, because a miss shows up two ways:
89 1. Clean omission (a real hole in the timeline) ? `possible_missed_rally` (gap
90 heuristic).
91 2. Merged/inflated (the missed rally got swallowed into a neighbour's box, leaving
92 no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
93 the second rally inside it.
94 Neither is a guarantee ? they surface regions to review, they don't certify
95 completeness. (A future enhancement: a motion second-opinion over the raw video.)
96
97 ### What's the review sheet?
98 `_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
99 blank `VERIFIED` / `true_start_mmss` / `true_end_mmss` / `true_ending_reason` /
100 `true_last_shot_outcome` / `reviewer_comment` columns. Open it next to the video
101 (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
102 a delivery into trusted golden labels.

```

```

100     ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
101     They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
102     unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
103     last shot went (in / net / out / n_a). A shuttle that flew long is
104     `forced_error/unforced_error` + `out` ? both kept, no information lost.
105
106     ### How do I onboard a brand-new video?
107     1. Get fps into the intake manifest (we provide it to the vendor).
108     2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
109        sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
110     3. The same suite runs for every video, so quality stays consistent at scale.
111
112     ### How do I pull a single rally out to eyeball it?
113     The slicer (indexer):
114     ```bash
115     python -m backend.tools.rally_slicer --video match.mp4 \
116         --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
117     ```
118     Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
119
120     ### Does coarse frame sampling hurt boundaries?
121     Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122     padding, so we don't flag sampling density.
123
124     ### Will a non-commercial license block export?
125     The video is **kept** in the DB regardless of license, but the default `export` is
126     **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
127     from the eval/training set unless you pass `--include-noncommercial`. So if a video
128     "goes missing" downstream, check its `is_commercial` flag first. (The observability
129     report raises a `license_noncommercial` warning for exactly this case.)
130
131     ### What is the observability report? (*.report.json` / `*.report.md` / `*.summary.md`)
132     Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
133     per-video **observability report** (next to the video for `import-local`; into the
134     `--out-dir` / next to the canonical CSV otherwise):
135     - **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff
136       two runs to see what changed between deliveries.
137     - **`<id>.report.md`** ? the **technical** report: parse / validation / registration
138       stages plus **findings** (one per delivery issue, each citing this FAQ). Read this
139       to debug a delivery.
140     - **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the
141       rally count and coverage plus the plain-language list of rallies to review ? no
142       internal codes or severities.
143     This is in addition to the existing `<id>_report.md` / `_report.json` / `_review.csv`
144     (the human review sheet), which are unchanged.
145

```

32. assistant (2026-06-08T08:39:58.330Z)

Excellent! Now let me check what the export command actually produces:

33. user (2026-06-08T08:39:59.017Z)

(Bash completed with no output)

34. user (2026-06-08T08:40:01.480Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\batch.py:Phase 3 of the
collector->indexer bridge: walk the `manifest.json` emitted by
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py:
records progress in ``<cache>\manifest.json`` (atomic write). A reboot loses at
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_infer.py:_MANIFEST_FILE = "manifest.json"
C:\Users\avidu\Projects\badminton-highlight-indexer\run_phase3_eval.py:Walks the `manifest.json`

```

```
produced by sports-data-collector's `curate export`,
C:\Users\avidu\Projects\badminton-highlight-indexer\run_phase3_eval.py:    python
run_phase3_eval.py --manifest ../sports-data-collector/data/eval_export/manifest.json \
```

35. user (2026-06-08T08:40:03.485Z)

```
1      """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3      Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4      processes each video through a segmenter (so detections land in the indexer DB),
5      scores each against its ground-truth rally CSV, and prints per-video plus
6      corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8      Examples
9      -----
10     Free CPU baseline on a single video (validate the loop):
11         python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
12         --segmenter motion --only shuttlestet_1
13
14     Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15         python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
16         --json output/phase3_report.json
17
18     Re-score only (videos already processed), no re-segmentation:
19         python run_phase3_eval.py --manifest ../manifest.json --score-only
20     """
21
22     import os
23     import sys
24     import json
25     import time
26     import argparse
27     import logging
28
29     from dotenv import load_dotenv
30     load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32     from backend.infrastructure.database import Database
33     from backend.config import load_config
34     from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
35     # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36     from backend.pipeline.segmenters import segmenter_registry
37     from backend.eval.annotations import load_annotations
38     from backend.eval.harness import evaluate, format_report_text, report_to_dict
39     from backend.eval import batch
40
41     logger = logging.getLogger(__name__)
42
43
44     def build_parser() -> argparse.ArgumentParser:
45         p = argparse.ArgumentParser(
46             description="Batch process + evaluate a collector export manifest (Phase 3).",
47             formatter_class=argparse.RawDescriptionHelpFormatter,
48         )
49         p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
50         p.add_argument("--videos-root", default=None,
51             help="Root for resolving manifest relative local_paths "
52             "(default: the collector repo root, two levels above the
manifest).")
53         p.add_argument("--segmenter", default="motion",
54             help="Segmenter to run (default: motion ? free/CPU). "
55             "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
56         p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
```

```

default="auto",
57             help="Which prediction stage(s) to score (default: auto by
segmenter).")
58     p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
59     p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
60     p.add_argument("--only", action="append", default=None,
61                     help="Only this video_id (repeatable).")
62     p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
63     p.add_argument("--max-frames", type=int, default=None,
64                     help="Cap segmenter frames (fast validation).")
65     p.add_argument("--score-only", action="store_true",
66                     help="Skip segmentation; only score detections already in the DB.")
67     p.add_argument("--reprocess", action="store_true",
68                     help="Re-run segmentation even if the video already has segments.")
69     p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
70     p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
71     p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
72     return p
73
74
75     def main(argv=None) -> int:
76         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
77         args = build_parser().parse_args(argv)
78
79         manifest_path = os.path.abspath(args.manifest)
80         if not os.path.exists(manifest_path):
81             print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
82             return 2
83         manifest_dir = os.path.dirname(manifest_path)
84         videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
85
86         with open(manifest_path) as f:
87             manifest = json.load(f)
88         entries = manifest.get("eval_sets", [])
89
90         cfg = load_config() # typed AppConfig (same object the live pipeline feeds
segmenters)
91         db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
92         os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
93         db = Database(db_path)
94
95         stages = batch.default_stages_for(args.segmenter, args.stage)
96
97         # Resolve the segmenter class up front (unless purely scoring).
98         segmenter = None
99         if not args.score_only:
100             seg_cls = segmenter_registry.get(args.segmenter)
101             if not seg_cls:
102                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
103                       f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
104                 return 2
105             segmenter = seg_cls(db, cfg)
106
107         # Filter entries.
108         targets = []
109         for e in entries:
110             if args.only and e["video_id"] not in args.only:
111                 continue
112             targets.append(e)

```

```

113         if args.limit:
114             targets = targets[: args.limit]
115
116         per_video = []
117         skipped = []
118         print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
119
120         for e in targets:
121             vid = e["video_id"]
122             video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
123             csv_path = os.path.join(manifest_dir, e["csv"])
124
125             if not video_path or not os.path.exists(video_path):
126                 skipped.append((vid, "no video file"))
127                 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
128                 continue
129             if not os.path.exists(csv_path):
130                 skipped.append((vid, "no GT csv"))
131                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
132                 continue
133
134             video_id = get_file_md5(video_path)
135
136             # Segment unless score-only or already present (and not --reprocess).
137             if not args.score_only:
138                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
139                 if already and not args.reprocess:
140                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
141                 else:
142                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
143                     db.add_video(video_id, video_path, "processed", [])
144                     t0 = time.time()
145                     segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
146                     print(f"          -> {len(segs)} segments in {time.time() - t0:.0f}s")
147
148                     gt = load_annotations(csv_path)
149                     report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
150                     print(format_report_text(report, show_failures=args.show_failures))
151                     print()
152                     d = report_to_dict(report)
153                     d["collector_video_id"] = vid
154                     per_video.append(d)
155
156             agg = batch.aggregate(per_video, stages)
157             print(batch.format_aggregate(agg))
158             if skipped:
159                 print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
160
161             if args.json_out:
162                 with open(args.json_out, "w") as f:
163                     json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
"aggregate": agg, "per_video": per_video,
164                               "skipped": skipped}, f, indent=2)
165                     print(f"\nJSON report written to {args.json_out}")
166
167             return 0
168
169         return 0
170
171     if __name__ == "__main__":

```

```
172         raise SystemExit(main())
173
```

36. assistant (2026-06-08T08:40:05.678Z)

Now let me check what the actual manifest.json structure looks like:

37. user (2026-06-08T08:40:06.460Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py: entries =
manifest.get("eval_sets", [])
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py-
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py- cfg = load_config()
# typed AppConfig (same object the live pipeline feeds segmenters)
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py- db_path = args.db or
os.path.join(cfg.output.output_dir, cfg.output.db_name)
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py- os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py- db =
Database(db_path)
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py-
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py- stages =
batch.default_stages_for(args.segmenter, args.stage)
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py-
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py- # Resolve the
segmenter class up front (unless purely scoring).
C:\Users\avidu\Projects\badminton-highlight-indexer/run_phase3_eval.py- segmenter = None
```

38. user (2026-06-08T08:40:08.834Z)

```
1      """Batch evaluation across a collector export manifest.
2
3      Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
4      sports-data-collector's `curate export`, (optionally) process each video through
5      a segmenter so detections land in the indexer DB, then score every video's
6      detected rallies against its ground-truth CSV and aggregate the results.
7
8      This module holds the *pure* pieces (path resolution, stage selection, metric
9      aggregation) so they can be unit-tested without touching video files, the GPU,
10     or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
11     """
12
13     import os
14     from typing import Dict, List, Optional
15
16
17     # Segmenters that only ever populate the `final` play_segments table (no raw
18     # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
19     _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}
20
21
22     def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
23         """Resolve a manifest `local_path` to an absolute path.
24
25         The collector stores paths like ``./data/videos\\shuttleaset_1.mp4`` relative
26         to its own repo root and with Windows separators. Normalise separators, strip
27         a leading ``./``, and resolve relative paths against `videos_root` (the
28         collector root). Returns None for a missing/empty path.
29         """
30         if not local_path:
31             return None
32         p = local_path.replace("\\", "/")
33         if p.startswith("./"):
34             p = p[2:]
```

```

35     # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
36     # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
37     # This keeps collector manifest paths working in tests and mixed environments.
38     if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
39         return os.path.normpath(p)
40     return os.path.normpath(os.path.join(videos_root, p))
41
42
43 def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
44     """Pick which prediction stages to score.
45
46     ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
47     but only ``final`` for segmenters that don't emit candidates. An explicit
48     request ('candidates' | 'final' | 'both') overrides the heuristic.
49     """
50     if requested == "both":
51         return ["candidates", "final"]
52     if requested in ("candidates", "final"):
53         return [requested]
54     # auto
55     if segmenter in _FINAL_ONLY_SEGMENTERS:
56         return ["final"]
57     return ["candidates", "final"]
58
59
60 def _safe_div(num: float, den: float) -> float:
61     return num / den if den else 0.0
62
63
64 def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
65     """Aggregate per-video stage scores into corpus-level metrics.
66
67     `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
68     ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
69     compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
70     a TP-weighted mean IoU, plus the macro mean of per-video F1.
71     """
72     out: Dict[str, dict] = {}
73     for stage in stages:
74         tp = fp = fn = 0
75         iou_weighted = 0.0
76         f1s: List[float] = []
77         n_videos = 0
78         for rep in per_video:
79             sr = rep.get("stages", {}).get(stage)
80             if not sr:
81                 continue
82             s = sr["score"]
83             tp += s["true_positives"]
84             fp += s["false_positives"]
85             fn += s["false_negatives"]
86             iou_weighted += s["mean_iou"] * s["true_positives"]
87             f1s.append(s["f1"])
88             n_videos += 1
89         precision = _safe_div(tp, tp + fp)
90         recall = _safe_div(tp, tp + fn)
91         out[stage] = {
92             "videos": n_videos,
93             "tp": tp, "fp": fp, "fn": fn,
94             "micro_precision": precision,
95             "micro_recall": recall,
96             "micro_f1": _safe_div(2 * precision * recall, precision + recall),
97             "macro_f1": _safe_div(sum(f1s), len(f1s)),
98             "mean_iou": _safe_div(iou_weighted, tp),
99         }

```


100 return out

39. assistant (2026-06-08T08:40:11.033Z)

Now let me find where the manifest.json is actually created in the collector:

40. user (2026-06-08T08:40:11.642Z)

```
C:\Users\avidu\Projects\sports-data-collector/src/cli/curate.py:
`<out_dir>/manifest.json` that pairs each CSV with its video file. The
C:\Users\avidu\Projects\sports-data-collector/src/cli/curate.py:          "eval_sets":
manifest,
C:\Users\avidu\Projects\sports-data-collector/src/cli/curate.py:          manifest_path =
os.path.join(out_dir, "manifest.json")
C:\Users\avidu\Projects\sports-data-collector/src/cli/curate.py:
help="Output directory for per-video CSVs and manifest.json")
C:\Users\avidu\Projects\sports-data-collector/tests/test_cli.py:      manifest = out_dir /
"manifest.json"
```

41. user (2026-06-08T08:40:15.497Z)

```
1      import os
2      import sys
3      import time
4      import json
5      import csv
6      import argparse
7      from typing import List, Dict, Any, Tuple
8      from tabulate import tabulate
9
10     from src.core.config import load_config
11     from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12     from src.core.db import SportsDatabase
13     from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14     from src.core.schemas.badminton import BadmintonRally
15     from src.core.schemas.tennis import TennisPoint
16     from src.core.schemas.cricket import CricketDelivery
17     from src.core.schemas.pickleball import PickleballRally
18     from src.core.schemas.tabletennis import TableTennisRally
19     from src.core.validator import DatasetValidator
20     from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23     class SportsDataCLI:
24         def __init__(self, config_path: str = "config/config.yaml"):
25             self.config_path = config_path
26             self.config = self._load_config()
27
28             # Resolve DB path
29             db_path = self.config.get("database_path", "data/sports_metadata.db")
30             self.db = SportsDatabase(db_path)
31             self.validator = DatasetValidator(config_path)
32
33         def _load_config(self) -> Dict[str, Any]:
34             if not os.path.exists(self.config_path):
35                 print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36             return load_config(self.config_path)
37
38         def status(self):
39             """Displays status summary of all videos in the database."""
40             # Query total count by status via the public DB read API.
41             rows = self.db.get_status_counts() # list of (sport, status, cnt)
```

```

42
43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
57 {self.config.get('supported_sports')}")
58             sys.exit(1)
59
60         # Validate video path
61         if not os.path.exists(args.video_path):
62             print(f"Warning: Local video path '{args.video_path}' does not exist on
63 disk, but continuing registration.")
64
65         # Validate annotation file
66         if not os.path.exists(args.annotations_path):
67             print(f"Error: Annotations file '{args.annotations_path}' not found.")
68             sys.exit(1)
69
70         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
71         video_id = args.video_id or match_name.lower().replace(" ", "_")
72
73         # Parse license
74         try:
75             license_type = LicenseType(args.license)
76         except ValueError:
77             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
78 (non-commercial).")
79             license_type = LicenseType.UNKNOWN
80
81         is_commercial = self.validator.check_license_commercial(license_type)
82
83         # 1. Parse Annotations
84         annotations = []
85         try:
86             annotations = self._parse_local_annotations(sport_str, video_id,
87 args.annotations_path)
88         except Exception as e:
89             print(f"Error parsing annotations: {e}")
90             sys.exit(1)
91
92         # 2. Run Validation
93         is_valid = True
94         errors = []
95         if sport_str == "badminton":
96             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
97         elif sport_str == "tennis":
98             is_valid, errors = self.validator.validate_tennis_points(annotations)
99         elif sport_str == "cricket":
100             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
101         elif sport_str == "pickleball":
102             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
103         elif sport_str == "tabletennis":
104             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
105
106         if not is_valid:

```

```

103         print(f"Error: Annotations failed validation!")
104     for err in errors:
105         print(f" - {err}")
106     sys.exit(1)
107
108 # 3. Create Video Metadata
109 video_meta = VideoMetadata(
110     video_id=video_id,
111     sport=SportType(sport_str),
112     video_url=None,
113     local_path=args.video_path,
114     license_type=license_type,
115     license_url=None,
116     is_commercial=is_commercial,
117     status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118     match_name=match_name,
119     fps=args.fps,
120     resolution=args.resolution
121 )
122
123 # 4. Insert into Database
124 self.db.insert_video(video_meta)
125
126 count = 0
127 if sport_str == "badminton":
128     count = self.db.insert_badminton_rallies(annotations)
129 elif sport_str == "tennis":
130     count = self.db.insert_tennis_points(annotations)
131 elif sport_str == "cricket":
132     count = self.db.insert_cricket_deliveries(annotations)
133 elif sport_str == "pickleball":
134     count = self.db.insert_pickleball_rallies(annotations)
135 elif sport_str == "tabletennis":
136     count = self.db.insert_tabletennis_rallies(annotations)
137
138 print(f"\nSuccessfully imported local video '{match_name}':")
139 print(f" - Video ID: {video_id}")
140 print(f" - Sport: {sport_str}")
141 print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142 print(f" - Registered {count} annotated segments.\n")
143
144 # Per-video observability report (next to the video). Never raises.
145 from src.reporting import emit_import_report
146 src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147 report_paths = emit_import_report(
148     video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149     errors=[], count=count, sport=sport_str, source_format=src_fmt,
150     fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151 )
152 if report_paths:
153     print(f" - Observability report: {report_paths.get('technical_md')}")
154     print(f" - Customer summary      : {report_paths.get('customer_md')}\n")
155
156 def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
157     """Parses simple JSON or CSV annotations files."""
158     parsed_items = []
159     ext = os.path.splitext(filepath)[1].lower()
160
161     if ext == '.json':
162         with open(filepath, 'r', encoding='utf-8') as f:
163             data = json.load(f)

```

```

164
165         if not isinstance(data, list):
166             raise ValueError("JSON file must contain a list of objects.")
167
168         # Uses the same safe casts as the CSV path: a missing/blank required
169         # time raises a clear ValueError (caught by the caller) instead of a
170         # bare KeyError, and optional ints degrade to sensible defaults.
171         for idx, obj in enumerate(data):
172             if sport == "badminton":
173                 parsed_items.append(BadmintonRally(
174                     video_id=video_id,
175                     rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                     start_time=_safe_float(obj.get("start_time")),
177                     end_time=_safe_float(obj.get("end_time")),
178                     score_before=obj.get("score_before"),
179                     score_after=obj.get("score_after"),
180                     shots_count=_safe_int(obj.get("shots_count"), None),
181                     ending_reason=obj.get("ending_reason"),
182                     last_shot_outcome=obj.get("last_shot_outcome")
183                 ))
184             elif sport == "tennis":
185                 parsed_items.append(TennisPoint(
186                     video_id=video_id,
187                     point_number=_safe_int(obj.get("point_number"), idx + 1),
188                     start_time=_safe_float(obj.get("start_time")),
189                     end_time=_safe_float(obj.get("end_time")),
190                     set_score=obj.get("set_score"),
191                     game_score=obj.get("game_score"),
192                     server=obj.get("server"),
193                     ending_type=obj.get("ending_type")
194                 ))
195             elif sport == "cricket":
196                 parsed_items.append(CricketDelivery(
197                     video_id=video_id,
198                     over=_safe_int(obj.get("over"), 0),
199                     ball=_safe_int(obj.get("ball"), 0),
200                     start_time=_safe_float(obj.get("start_time")),

```

42. user (2026-06-08T08:40:18.297Z)

```

440:     def export_eval_set(self, args):

```

43. user (2026-06-08T08:40:20.686Z)

```

440     def export_eval_set(self, args):
441         """Export curated annotations as indexer-ready eval/training sets.
442
443         Emits, per qualifying video, a `

```

```

461
462 manifest = []
463 skipped_noncomm = 0
464 skipped_empty = 0
465
466 for sport_str in sport_strs:
467     try:
468         sport = SportType(sport_str.lower())
469     except ValueError:
470         print(f"Warning: skipping unknown sport '{sport_str}'.")
471         continue
472
473     videos = []
474     for status in statuses:
475         videos.extend(self.db.get_videos_by_status(status, sport))
476
477     for video in videos:
478         if commercial_only and not video.is_commercial:
479             skipped_noncomm += 1
480             continue
481
482         intervals = self.db.get_segment_intervals(video.video_id, sport)
483         if not intervals:
484             skipped_empty += 1
485             continue
486
487         sport_dir = os.path.join(out_dir, sport.value)
488         os.makedirs(sport_dir, exist_ok=True)
489         csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490         with open(csv_path, "w", newline="", encoding="utf-8") as f:
491             writer = csv.writer(f)
492             writer.writerow(["start", "end"])
493             for start, end in intervals:
494                 writer.writerow([start, end])
495
496         manifest.append({
497             "video_id": video.video_id,
498             "sport": sport.value,
499             "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500             "local_path": video.local_path,
501             "video_url": video.video_url,
502             "match_name": video.match_name,
503             "fps": video.fps,
504             "resolution": video.resolution,
505             "license_type": video.license_type.value,
506             "is_commercial": video.is_commercial,
507             "status": video.status.value,
508             "num_segments": len(intervals),
509         })
510
511 os.makedirs(out_dir, exist_ok=True)
512 manifest_doc = {
513     "commercial_only": commercial_only,
514     "statuses": [s.value for s in statuses],
515     "total_videos": len(manifest),
516     "total_segments": sum(m["num_segments"] for m in manifest),
517     "eval_sets": manifest,
518 }
519 manifest_path = os.path.join(out_dir, "manifest.json")
520 with open(manifest_path, "w", encoding="utf-8") as f:
521     json.dump(manifest_doc, f, indent=2)
522
523 print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524 if manifest:
525     summary = [[m["sport"], m["video_id"], m["num_segments"],

```

```

526                 "YES" if m["is_commercial"] else "NO",
527                 os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
528             for m in manifest]
529         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530         print(f"\nVideos exported : {len(manifest)}")
531         print(f"Total segments : {manifest_doc['total_segments']}")
532         if skipped_noncomm:
533             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534         if skipped_empty:
535             print(f"Skipped (no annotations) : {skipped_empty}")
536         print(f"Manifest : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
540
541         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
542         attributes) on the ingestion side: each rally's time is recomputed from
543         its frame range at the video's true fps, and content defects (overlaps,
544         zero-duration, off-vocabulary endings, coarse sampling) are written to an
545         issues report rather than silently "fixed". The emitted CSV imports via
546         `import-local`; any blocker issue must be resolved first, since the
547         importer's validator rejects overlapping / zero-duration rallies.
548         """
549         from src.parsers.cvat.rally_cvat import (
550             convert, write_canonical_csv, render_issue_report,
551         )
552         try:
553             result = convert(args.input, fps=args.fps,
554                             video_path=args.video_path, step=args.step)
555         except (FileNotFoundError, ValueError) as e:
556             print(f"Error: {e}")
557             sys.exit(1)
558
559         write_canonical_csv(result.records, args.out)
560
561         # Observability report next to the canonical CSV. Never raises.
562         from src.reporting import emit_delivery_report
563         src_name = os.path.splitext(os.path.basename(args.input))[0]
564         dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
565         emit_delivery_report(
566             out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
567             records=result.records, issues=result.issues, fps=result.fps,
568             video_path=args.video_path,
569             video_duration_s=dur, source_format="cvat", step=result.step,
570         )
571
572         report = render_issue_report(result, os.path.basename(args.input))
573         if args.issues:
574             with open(args.issues, "w", encoding="utf-8") as f:
575                 f.write(report)
576         print(report)
577         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
578         if result.n_blockers:
579             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
580                 f"this file until they are resolved (by the vendor or manual
review).")
581         else:
582             print("\n No blockers. Next:")
583             print(f"    python -m src.cli.curate import-local --sport <sport> "
584                   f"--video-path <video> --annotations-path {args.out} --fps

```

```
{result.fps:g}")
584
585         def validate_delivery(self, args):
586             """Run the full validation suite on a rally delivery and emit a report.
587
588             Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
589             CSV (--csv). Writes, into --out-dir:
```

44. assistant (2026-06-08T08:40:23.141Z)

Perfect! Now I have all the information I need. Let me also check if there's any documentation about partial labels or labeled windows in the collector:

45. user (2026-06-08T08:40:23.923Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/config/loader.py:
logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/eval_partial_labels.py: """Evaluate the rally pipeline against PARTIAL human
golden labels.
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/eval_partial_labels.py: ending_reason, sport). While a video is only
partially tagged, the labels cover a
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/eval_partial_labels.py:
python -m backend.eval.eval_partial_labels \
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/eval_partial_labels.py:
print("=== Pipeline vs PARTIAL human labels ===")
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py:
f"Clamping to {video_duration:.2f}s (end padding partially applied).")
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_metrics.py: def
test_iou_partial():
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_partial_labels.py: """Eval
against PARTIAL human labels.
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_partial_labels.py: partially
tagged, the detector's correct rallies in the un-labeled tail must NOT count
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_partial_labels.py: from
backend.eval import eval_partial_labels as ep
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_partial_labels.py: def
_write_partial_labels(path):
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_partial_labels.py:     labels
= _write_partial_labels(tmp_path / "p.csv")
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_partial_labels.py:     labels
= _write_partial_labels(tmp_path / "p.csv")
```

46. assistant (2026-06-08T08:40:26.218Z)

Now let me compile all the findings. Let me create a comprehensive summary with anchors:

47. user (2026-06-08T08:40:55.141Z)

Structured output provided successfully

48. assistant (2026-06-08T08:41:03.209Z)

Summary

I've completed a thorough analysis of the collector's output contract with the indexer's eval/training pipeline. Here are the key findings:

****CSV Contract (Ground Truth)****

- ****Minimal columns required****: `start,end` (decimal seconds)
- ****Optional header****: `start` or `start\_time` (case-insensitive)
- ****Format****: One rally per row; blank/comment lines ignored

- **Time format**: Decimal seconds (float) preferred; colon-separated `MM:SS`/`HH:MM:SS` also supported in some harnesses

Manifest.json Contract (created by `sports-data-collector curate export`)

Each entry contains:

- `video\_id`, `sport`, `csv` (relative path to ground-truth CSV)
- `local\_path` (file system path, used for MD5 keying across repos)
- `fps` (required for trajectory windowing and feature normalization)
- `frame\_width` (required for WASB velocity normalization)
- `resolution`, `license\_type`, `is\_commercial`, `status`, `num\_segments` (metadata)

IoU Matching & Labeled Windows

- **Greedy one-to-one matching** at `iou\_threshold` (default 0.5)
- Highest-IoU pairs claimed first; each prediction/GT used at most once
- **Labeled-window concept** (for partial labels): restrict evaluation to `[window\_start, window\_end)` interval
 - **NOT representable** in the current export format? applied only as CLI args to `eval\_partial\_labels.py`
 - Predicted segments outside the window are dropped without affecting metrics
 - GT assumed to cover a PREFIX of video (every rally up to window_end is labeled)

Key Metadata Gaps

1. **Rally metadata** (ending_reason, shots_count, score) are **validated but discarded**? only `(start\_time, end\_time)` flow downstream
2. **Partial-label support** is NOT persisted in the manifest? it's an operational harness feature, not a data contract
3. **Trajectory CSV path** is NOT in the manifest? supplied separately as CLI arg; no guarantee of co-location
4. **Multi-sport mixing** is technically possible but not validated or documented in contracts

The contract is **temporal-only** (no frame-level shuttle coordinates needed for segmentation) and **resolution-invariant** (features normalized by fps and frame_width, not absolute pixels).